Registers and Shift Registers

Contemporary Digital Electronic Systems — Chapter 11

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Data Registers
3. Shift Registers
4. Shift-Register Counters
5. Accumulators
6. Summary

Why This Chapter Matters

A flip-flop stores one bit. In practice we store, move and transform groups of bits: a 32-bit word, a 16-bit instruction, a stream of serial data. Grouping flip-flops into registers, and especially shift registers, gives building blocks for serial links, pipelines and every sequential module that follows. Two kinds of counter appear here, shift-register and accumulator counters; flip-flop-based counters follow in Chapter 12.

After this chapter, you can:

1. Draw an *n*-bit parallel-load register and describe its controls.
2. Distinguish SISO, SIPO, PISO and PIPO shift registers and give an application of each.
3. Design a shift register with load, shift-left, shift-right and hold modes.
4. Explain serial–parallel conversion.
5. Recognise ring and Johnson counters as shift-register configurations.

© Muním Zabidi

An Opening Question

How does a laptop talk to a USB device through a connector with only a handful of wires, when thousands of bytes must be exchanged?

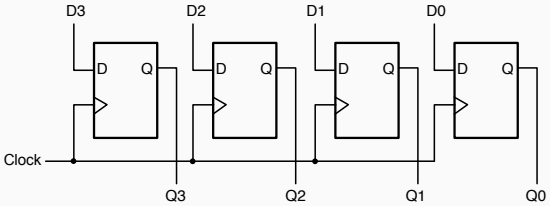
The key idea is serial–parallel conversion with shift registers.

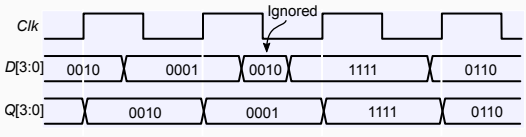
© Muním Zabidi

Table of Contents

1. Why This Chapter
2. Data Registers
3. Shift Registers
4. Shift-Register Counters
5. Accumulators
6. Summary

Basic Data Register

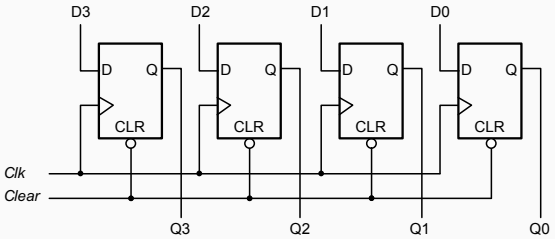


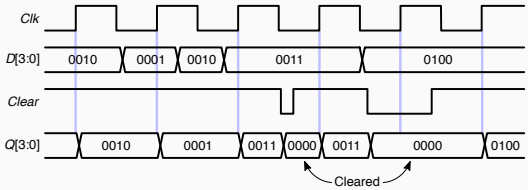


- A register holds binary information while it awaits processing or after it is produced.
- Four D flip-flops share one clock: on every rising edge the *D* inputs are stored. Most registers use positive-edge-triggered D flip-flops.

© Muním Zabidi

Register with Clear

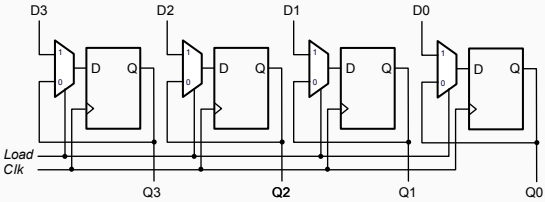


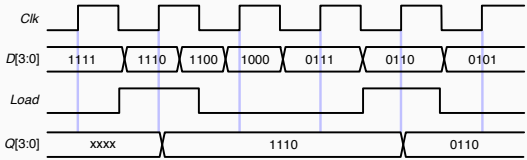


- Registers must start from a known state (start-up, error recovery), so they get a reset input.
- An active-low, asynchronous clear forces all flip-flops to 0 immediately, with no clock pulse needed.

© Munim Zabidi

Register with Load Enable





- A mux at each flip-flop input, controlled by `Load`.
- `Load` = 1: new data enters on the next rising edge. `Load` = 0: the flip-flop's own output is fed back, so the register holds its value.
- The register updates only when new data is available.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Data Registers

3 **Shift Registers**

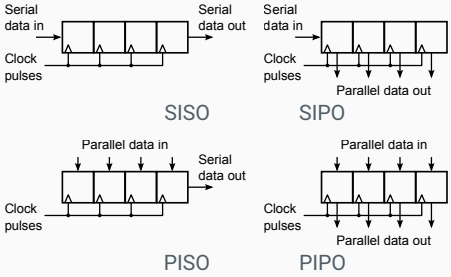
4 Shift-Register Counters

5 Accumulators

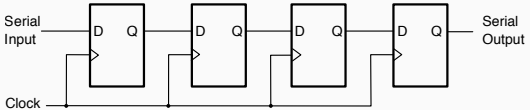
6 Summary

Four Types of Shift Register

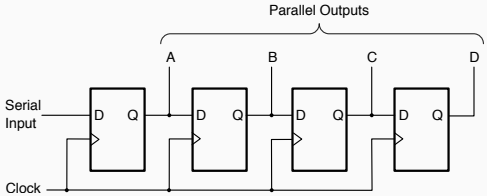
- A shift register passes data from stage to stage on each clock pulse: storage, transfer, delay, serial–parallel conversion.
- SISO: serial in, serial out.
- SIPO: serial in, parallel out.
- PISO: parallel in, serial out.
- PIPO: parallel in, parallel out.



SISO and SIPO Logic Diagrams



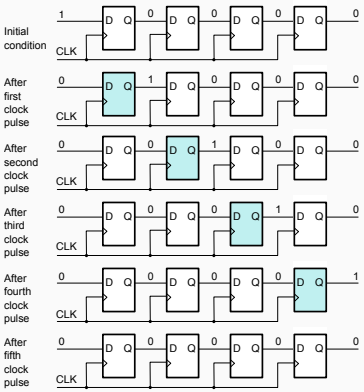
SISO



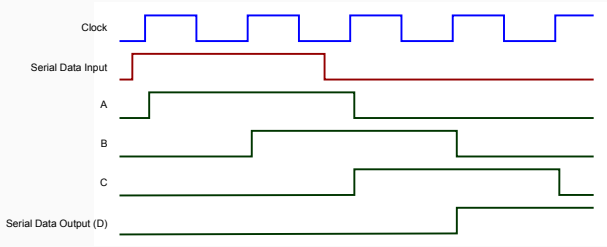
SIPO

- Both shift data one stage per clock. A SISO gives only the last bit; a SIPO exposes every stage.

SIPO Operation



1. Apply a bit to the serial input.
2. Apply a clock pulse.
3. The bit is stored in the first flip-flop.
4. Every stage shifts one position right.
5. Read the parallel outputs.
6. Repeat until all bits are loaded.



© Muním Zabidi

Example: Loading 1011 into a SIPO Register

Stages A, B, C, D (A nearest the input), starting at 0000; input `1011`, MSB first.

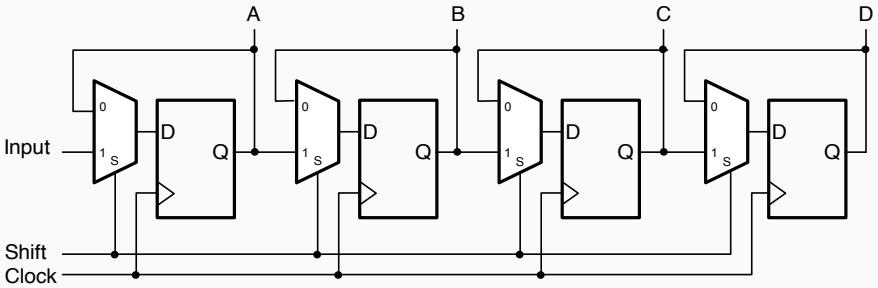
Pulse	In	A	B	C	D
1	1	1	0	0	0
2	0	0	1	0	0
3	1	1	0	1	0
4	1	1	1	0	1

- After four pulses $ABCD = 1101$.
- Reading from D (first bit in) to A (last bit in) recovers `1011`.

Uses

SISO: delay line. SIPO / PISO: serial–parallel interface. PIPO: temporary storage of a whole word.

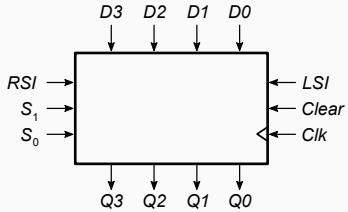
Shift Register with Enable



- A mux at each flip-flop input, controlled by a common `Shift` signal. `Shift` = 1: shift each clock. `Shift` = 0: no shifting.
- Larger muxes allow more input sources, including external data: the route to PISO and PIPO operation.

© Muním Zabidi

Universal Shift Register

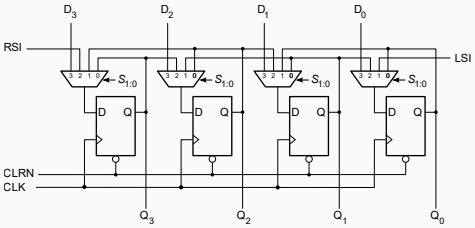


S_1	S_0	Function
0	0	Hold
0	1	Shift left
1	0	Shift right
1	1	Load

- Serial-to-serial, serial-to-parallel, parallel-to-serial, parallel-to-parallel; left and right shifts.
- Shift left: $Q_3Q_2Q_1 \leftarrow Q_2Q_1Q_0$, $Q_0 \leftarrow LSI$.
- Shift right: $Q_2Q_1Q_0 \leftarrow Q_3Q_2Q_1$, $Q_3 \leftarrow RSI$.
- Left shift is like multiplication by 2; right shift like division by 2.

© Munim Zabidi

Universal Shift Register: Schematic and Timing



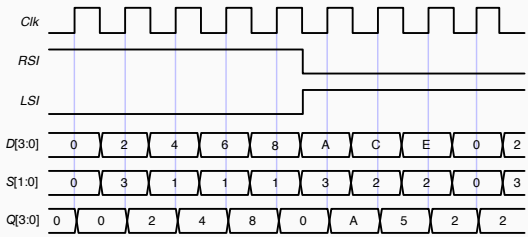


Table of Contents

1 Why This Chapter

2 Data Registers

3 Shift Registers

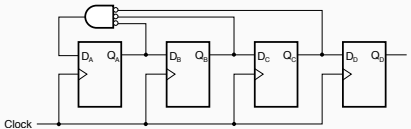
4 **Shift-Register Counters**

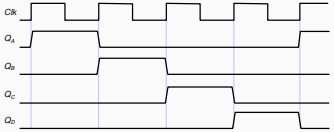
5 Accumulators

6 Summary

Ring Counter

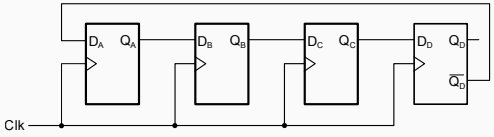
- The last stage feeds the first: a single 1 circulates. 4-bit sequence:
  $1000 \to 0100 \to 0010 \to 0001 \to 1000 \to \cdots$
- n flip-flops give only n valid states (versus 2^n for a binary counter), but each state is trivially decoded.
- It must be initialised: set one flip-flop and clear the rest (preset on the first, clear on the others), or detect all-zero and reload ($D_A = \overline{A}\,\overline{B}\,\overline{C}$).





Johnson Counter

- Also called twisted-ring or Möbius counter: feeds the inverted last output to the first.
- n flip-flops give $2n$ states.
- Only one bit changes per clock, and each state is easier to decode than a ring counter's.
- Uses $2n$ of 2^n states: invalid power-up states form separate loops and do not self-correct without added logic or a reset.

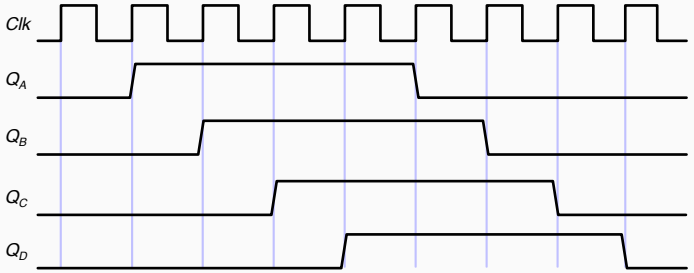


Clock	A	B	C	D
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
8	0	0	0	0

Back to 0000 after $2n = 8$ pulses.

© Muním Zabidi

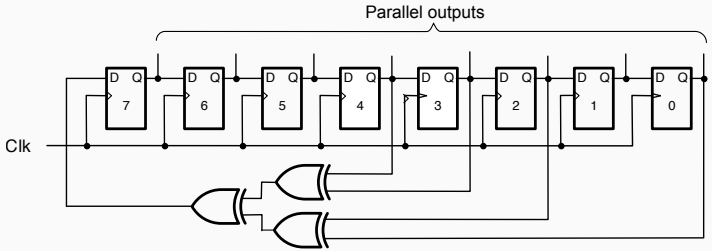
Johnson Counter: Timing



- Eight distinct states with one bit changing at each step, which helps reduce decoding errors.

© Muním Zabidi

Linear-Feedback Shift Register (LFSR)



- Selected stage outputs (the taps) are XORed and fed back to the first stage.
- With the right taps, an n-bit LFSR cycles through all $2^n - 1$ non-zero states: a pseudo-random sequence (testing, cryptography, communications).
- 8 bits: taps 4, 3, 2, 0 give 255 states. The all-zero state is forbidden: it never changes.

LFSR Tap Table and Example

Stages	Length	Taps
2	3	1,0
3	7	1,0
4	15	1,0
5	31	2,0
6	63	1,0
7	127	1,0
8	255	4,3,2,0
9	511	4,0
10	1023	3,0
12	4095	6,4,1,0
16	65535	12,3,1,0

Load `00000001`; $F = Q_4 \oplus Q_3 \oplus Q_2 \oplus Q_0$ enters Q_7.

Clock	State $Q_7 \ldots Q_0$	Dec	F
0	0000 0001	1	1
1	1000 0000	128	0
2	0100 0000	64	0
3	0010 0000	32	0
4	0001 0000	16	1
5	1000 1000	136	1
6	1100 0100	196	1

Table of Contents

1 Why This Chapter

2 Data Registers

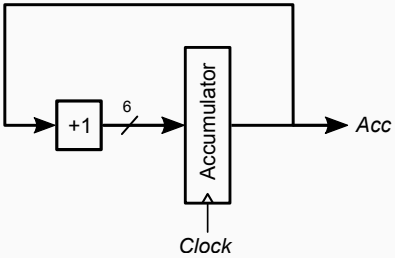
3 Shift Registers

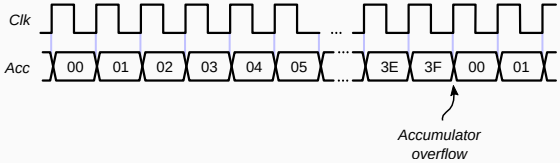
4 Shift-Register Counters

5 **Accumulators**

6 Summary

Accumulator-Based Counters

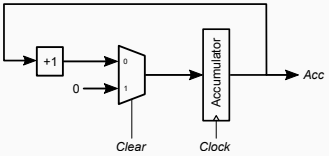




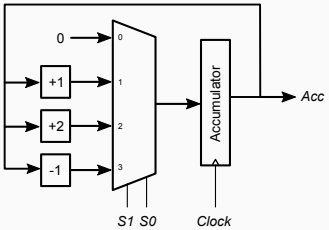
- An accumulator is a register whose output is fed back through combinational logic: results build up over clock cycles.
- A register plus an incrementer is a counter: this 6-bit one counts 00 to 3F and wraps to 00.
- Easy to design (register width sets the count size); speed is set mainly by the adder delay.

© Muním Zabidi

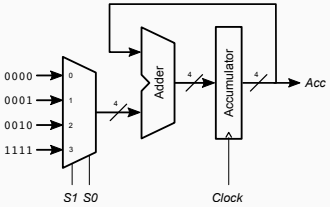
Resettable and Multifunction Counters



Clear asserted: load 0



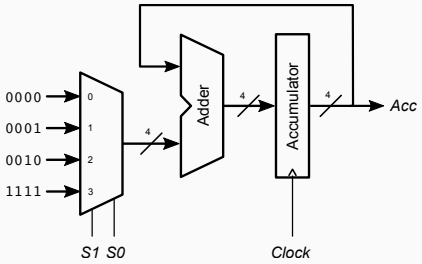
Clear, +1, +2, —1



One adder, mux picks the operand

- A mux in the feedback path changes the counting sequence. The single-adder version needs less hardware for the same functions.

Example: Multifunction Counter

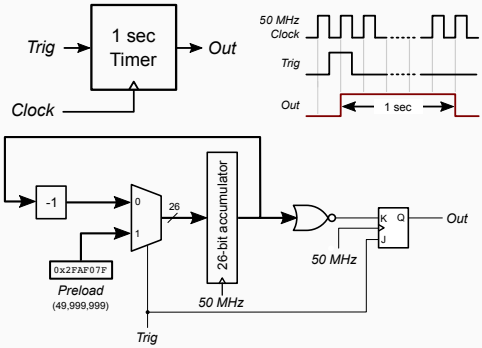


The single-adder multifunction counter

S_1	S_0	Function
0	0	Hold
0	1	Add 1
1	0	Add 2
1	1	Subtract 1

Clk	S_1	S_0	Function	Q
0	–	–	initial	0000
1	0	1	Add 1	0001
2	0	1	Add 1	0010
3	1	0	Add 2	0100
4	1	1	Subtract 1	0011

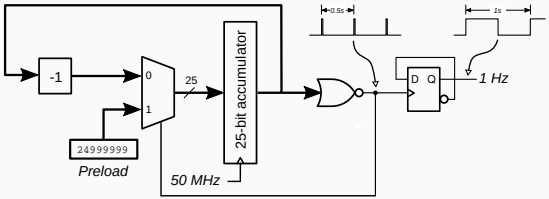
- Start at $Q = 0000$; control sequence $S_1S_0 = 01, 01, 10, 11$.
- After four pulses $Q = 0011_2 = 3$.
- The function can change on every pulse; one adder serves all operations.



- A timer gives a pulse of set duration: a one-shot (monostable).
- 1 s at 50 MHz: on *Trig*, load a 26-bit accumulator with $50\,000\,000 - 1$ and set the JK flip-flop. It counts down; at zero a NOR gate resets the flip-flop.

$$\text{Preload} = \text{Delay} \times \text{Frequency} - 1$$

Prescaler



- A prescaler derives a slow time base from a fast clock.
- 25-bit counter loaded with 25 000 000 − 1 and decremented each cycle. At zero a NOR gate pulses and reloads it: one pulse every 0.5 s, i.e. 2 Hz.
- The pulses are very asymmetric. A D flip-flop in toggle mode gives a symmetric 1 Hz, 50% duty cycle.
- A *programmable* prescaler takes its preload from a control circuit.

Timers and prescalers need big counters. When timing is not critical, a 555 timer (monostable or astable) is cheaper.

© Muním Zabidi

Back to the Opening Question

Serial links use shift registers

A USB-like link sends bits one after another on a few wires. The sender loads a word in parallel and shifts it out serially (PISO). The receiver shifts the bits in (SIPO) and reads the whole word in parallel. Two or three wires carry data and clock, yet thousands of bytes are exchanged.

Table of Contents

1 Why This Chapter

2 Data Registers

3 Shift Registers

4 Shift-Register Counters

5 Accumulators

6 **Summary**

Summary

- An *n*-bit data register is *n* flip-flops sharing a clock and, optionally, a clear and a load enable.
- A shift register shifts the stored word one position per clock. Its main use is serial–parallel conversion (SIPO, PISO).
- A ring counter recirculates a single 1; a Johnson counter feeds back the complement and visits $2n$ states.
- An LFSR XORs selected taps back to the input and generates a pseudo-random sequence.
- An accumulator with an incrementer is a counter; a timer or prescaler is a loadable down-counter with a terminal-count detector.

Looking ahead

Chapter 12 builds counters directly from flip-flops, circuits that produce an ordered sequence of states for counting, dividing and timing.

© Muním Zabidi